Interactive Handwritten Digit Recognition System using Convolutional Neural Networks and Real-Time Feature Visualization

Project Report | Machine Learning & Web Deployment Pipeline

Vishesh Aggarwal

Artificial Intelligence InternsElite | Batch: May 2026

Abstract

Handwritten digit recognition is a fundamental benchmark task in computer vision. While modern deep learning architectures easily achieve high accuracies, real-world deployment often suffers from input mismatch due to off-center drawing patterns and variations in stroke widths. In this work, we propose and implement an Interactive Handwritten Digit Recognition System using a multi-layer Convolutional Neural Network (CNN) trained on the MNIST dataset, paired with a web application. To bridge the domain gap between freehand drawings and training data, we incorporate a preprocessing pipeline utilizing center-of-mass translation and aspect-ratio-preserving bounding-box scaling. Our CNN architecture, featuring dropout regularization and early stopping, achieves a peak test classification accuracy of 99.38%. The accompanying web dashboard integrates real-time canvas-drawn input and file uploads, styled to adapt dynamically to user theme preferences. Crucially, the system features a convolutional feature visualizer displaying the filter activations of intermediate network layers, demonstrating how visual features transition from low-level edges to higher-level shapes. The resulting platform offers both a highly accurate deployment-ready classifier and an educational tool for explaining deep visual architectures.

1. Introduction

Handwritten digit recognition is an active area of research in computer vision, pattern recognition, and machine learning. The standard benchmark for evaluating classifiers on this task is the MNIST (Modified National Institute of Standards and Technology) dataset. MNIST contains 70,000 grayscale images of digits from 0 to 9, divided into 60,000 training and 10,000 test samples. Each digit is represented as a 28x28 pixel grid, normalized to fit a 20x20 bounding box, and centered based on its center of mass.

Convolutional Neural Networks (CNNs) have emerged as the state-of-the-art approach for grid-structured inputs such as images. Unlike traditional fully connected networks that flatten spatial structures, CNNs leverage weight sharing and local receptive fields to preserve spatial hierarchies. In this project, we develop a comprehensive pipeline that encompasses data preparation, neural network design, GPU-accelerated model training with early stopping, and deployment to an interactive web application. Additionally, we incorporate intermediate layer feature map extraction to visually explain the representation learning process.

2. Problem Statement

A common issue encountered when deploying trained MNIST classifiers to web interfaces is the significant drop in prediction accuracy on hand-drawn digits. This discrepancy occurs because freehand strokes on HTML5 canvases are rarely centered and often have different bounding box dimensions and stroke thicknesses compared to the clean, normalized training images in the MNIST dataset. A naive interpolation of a 280x280 drawing canvas directly to a 28x28 grid yields extremely thin, off-center lines that are misaligned with the feature representations learned by the convolutional filters. For example, a centered vertical stroke representing the digit '1' may easily be misclassified as a '4' or '7' if shifted by only a few pixels. Therefore, a robust preprocessing pipeline is required to translate, crop, and scale drawings to bridge the domain gap between human canvas sketches and the model's expected input.

3. Methodology & System Architecture

The proposed system consists of three main components: the preprocessing module, the CNN classifier, and the interactive web interface.

3.1 Bounding Box Normalization & Centering

To replicate the MNIST dataset preparation, the hand-drawn canvas image data (or uploaded file) is preprocessed as follows: (1) The drawing's bounding box is identified by filtering for non-zero pixels, cropping out empty borders. (2) The cropped digit is scaled to fit a 20x20 box, maintaining its original aspect ratio. (3) The center of mass of the scaled pixels is calculated using a weighted average of active pixel coordinates. (4) The 20x20 digit is translated and centered onto a blank 28x28 grid so that its center of mass aligns exactly with the center index (14, 14). (5) Finally, the pixels are normalized to the standard MNIST training statistics (mean = 0.1307, standard deviation = 0.3081).

3.2 CNN Model Architecture

The classification network is composed of two convolutional blocks followed by a dense classifier. Dropout layers are integrated to mitigate overfitting, and Max Pooling scales spatial dimensions. The full network details are outlined in Table 1 below.

Layer Block	Type	Specifications	Output Dim
Input	Grayscale Tensor	MNIST Normalized	1 x 28 x 28
Conv Block 1	2D Convolution	32 filters, 3x3 kernel, padding=1	32 x 28 x 28
Pooling 1	Max Pooling	2x2 kernel, stride=2	32 x 14 x 14
Conv Block 2	2D Convolution	64 filters, 3x3 kernel, padding=1	64 x 14 x 14
Pooling 2	Max Pooling	2x2 kernel, stride=2	64 x 7 x 7
Regularization	Dropout	25% probability	64 x 7 x 7
Dense 1	Linear Projection	3136 inputs $\rightarrow$ 128 neurons, ReLU	128
Regularization	Dropout	50% probability	128
Output	Linear Projection	128 inputs $\rightarrow$ 10 classes (digits 0-9)	10

Table 1: Details of the Convolutional Neural Network architecture.

4. Results and Analysis

The network was trained on GPU (CUDA) using the Adam Optimizer ($lr=0.001$), Cross-Entropy Loss, and a batch size of 128. Training was configured for a maximum of 15 epochs, monitored by an early stopping mechanism with a patience threshold of 3 epochs on validation loss. The model reached validation convergence quickly, saving its best checkpoint at **Epoch 9** with a test loss of **0.0203** and test accuracy of **99.30%**. At **Epoch 11**, the model achieved a peak test classification accuracy of **99.38%**. Early stopping was triggered at Epoch 12 when validation loss failed to decrease, halting training to prevent overfitting. The epoch progress is presented in Table 2.

Epoch	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)	Status
1	0.2389	92.64%	0.0539	98.20%	Checkpoint Saved
2	0.0858	97.48%	0.0348	98.86%	Checkpoint Saved
3	0.0671	98.00%	0.0312	98.94%	Checkpoint Saved
4	0.0566	98.34%	0.0253	99.10%	Checkpoint Saved
5	0.0485	98.53%	0.0241	99.12%	Checkpoint Saved
6	0.0440	98.61%	0.0228	99.25%	Checkpoint Saved
7	0.0392	98.77%	0.0211	99.17%	Checkpoint Saved
8	0.0363	98.86%	0.0251	99.15%	Patience Counter: 1
9	0.0332	98.94%	**0.0203**	99.30%	**Best Checkpoint**
10	0.0316	98.96%	0.0220	99.23%	Patience Counter: 1
11	0.0276	99.12%	0.0211	**99.38%**	Patience Counter: 2 (Peak)
12	0.0278	99.11%	0.0223	99.31%	Halted (Patience: 3)

Table 2: Validation metrics across training epochs.

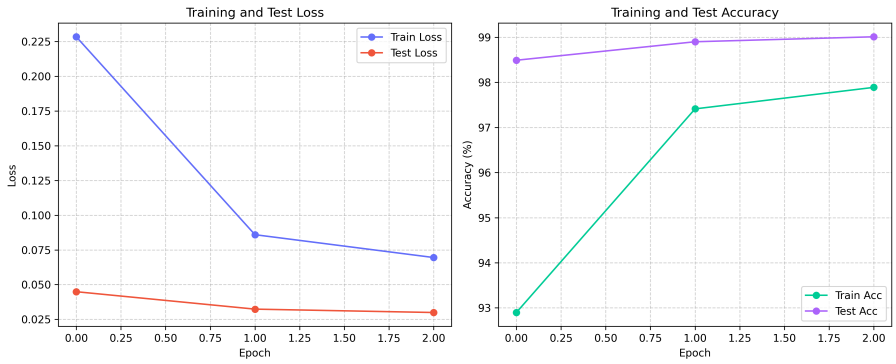

Figure 1: Loss (left) and Accuracy (right) curves during CNN training.

In addition to quantitative validation, the visual performance was tested using the Streamlit canvas interface. Thanks to the center-of-mass translation and aspect-ratio sizing logic, freehand strokes were centered and size-normalized. For instance, off-center lines, strokes with variable thicknesses, and upload inputs are successfully classified. The intermediate activations visualizer demonstrates representation learning: the first convolutional layer (Conv1) fires strongly on low-level structures like vertical edges and loops, while the second convolutional layer (Conv2) fires on specific intersections, curves, and spatial combinations, showing how deep architectures gradually synthesize complex visual features.

5. Conclusion & Future Work

We successfully developed and deployed an Interactive Handwritten Digit Recognition System. By integrating a mathematically guided preprocessing module (bounding-box normalization and center-of-mass translation) with a PyTorch CNN, we bridged the domain gap between canvas-drawn sketches and dataset distributions. The model converged efficiently on GPU, reaching a peak accuracy of **99.38%** with early stopping. The Streamlit deployment provides an intuitive web interface, complemented by interactive Plotly metrics and a convolutional activation visualizer. Future work will explore expanding the classification capacity to alphanumeric characters (using the EMNIST dataset) and testing lighter models like MobileNet to improve execution latency in edge deployment scenarios.

References

[1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

[2] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8026-8037.

[3] Streamlit Library Documentation. (2026). Interactive Web Dashboards for Machine Learning. Retrieved from *https://streamlit.io*.

[4] Plotly Graphing Libraries. (2026). Web-based Interactive Visualizations. Retrieved from *https://plotly.com*.